

Stochastic Baskets: Investigating the NCAA March Madness Tournament Using Stochastic Modeling

Nathan Bronson
Dept. of Mathematics
Harvey Mudd College
Claremont, CA, USA

Edward Donson
Dept. of Mathematics
Harvey Mudd College
Claremont, CA, USA

Tyler Headley
Dept. of Mathematics
Harvey Mudd College
Claremont, CA, USA

Matthew Shield
Dept. of Mathematics
Harvey Mudd College
Claremont, CA, USA

Abstract—The NCAA Division I men’s basketball tournament, commonly known as March Madness, offers an abundance of opportunities for stochastic modeling in a popular context. We propose techniques to find the most likely tournament outcome and build optimal tournament seedings. We also propose metrics to model the optimality of each solution. We address these problems with Markov chain Monte Carlo methods, including simulated annealing, basin hopping, and the Metropolis-Hastings algorithm, which allow us to efficiently search and sample from these vast spaces. Our models outperform several baselines in modeling the 2023 NCAA Tournament.

Index Terms—Markov chain Monte Carlo, Simulated Annealing, NCAA March Madness Tournament, Stochastic Modeling.

I. EXECUTIVE SUMMARY

The NCAA Division I men’s basketball tournament offers an abundance of opportunities for stochastic modeling. Commonly known as March Madness, the event is a central part of American sports culture. March Madness is a single elimination seeded bracket tournament that pits the top 64 NCAA Division I Men’s Basketball teams against one another in order to crown a champion.

Randomness is evident and optimization is necessary in nearly every part of the tournament. Games are unpredictable, optimal scheduling is nearly intractable, and both can significantly affect the tournament’s immense revenue generation. Studies like [1], [2], and [3] use statistical and stochastic models to model games, predict outcomes, and optimize scheduling. Each study emphasizes the utility of accurately modeling the tournament.

We focus our analysis on tournament outcomes and dynamics. Prior to the tournament, fans ritually fill out a bracket (like the one shown in Figure 1), predicting the outcomes of the tournament’s 63 games and often wagering money on correctly picking the most winners. Because filling out a bracket requires a fan to make 63 matchup decisions, there are a total of 2^{63} , or approximately 10^{18} , possible brackets. These matchups are notoriously unpredictable, so the chance of filling out a correct bracket is infinitesimal. Some fans, such as Georgia Tech professor Joel Sokol, try to improve their chances of predicting the outcome by modeling the tournament using statistical methods. With his LRMC model, described in [2], Sokol was able to predict three of the final four remaining teams for the 2025 March Madness tournament.

Prior to the tournament, the NCAA must also seed each of the 64 teams, ranking them from strongest to weakest. With its seeding, the NCAA seeks to minimize the number of “upsets”, or games won by the team with the worse seed. Finding the optimal seeding is even more difficult than predicting the outcome, with $64!$ (approximately 10^{89}) possibilities.

We use statistical modeling to address both the tournament outcome problem and the seeding problem.¹ We assume that our audience has already developed a model which predicts the probability of one team beating another in a matchup. It is easy to access reliable probabilities to specific matchup outcomes. For example, popular sports media outlets, like ESPN, publish odds, which can be converted into probabilities, for every NCAA basketball game played [4]. Even non-statistically-savvy sports fans build informal matchup probability models. Before a game, a fan might say “Team 1 is the favorite, but I wouldn’t be surprised if Team 2 pulls it off.” Formally quantifying these models to address every possible matchup enables a wide range of opportunities to harness and validate their outputs.

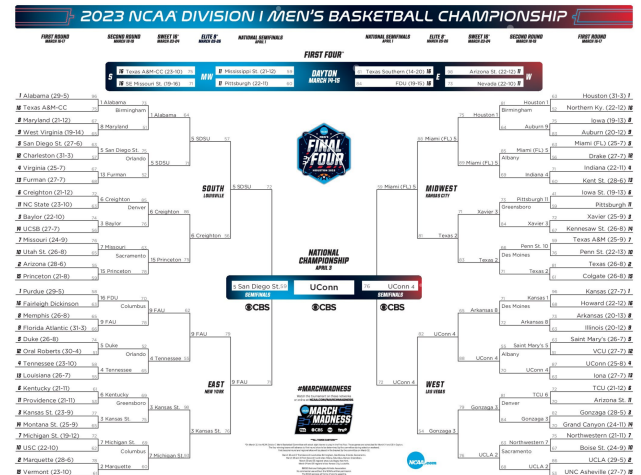


Figure 1: Completed bracket from the 2023 Men’s March Madness Tournament [5].

¹Code used to run experiments and generate figures can be found at <https://github.com/jhawgs/StochasticBaskets>.

Our solution to the tournament outcome problem caters to an audience of statistically savvy basketball fans and sports bettors. This audience wants to find the most likely tournament outcome with minimal computational requirements. Our solution to the seeding problem caters to the NCAA, which is responsible for creating seedings. Similar to the previous problem’s audience, the NCAA is interested in an optimal solution but has more relaxed computational requirements. While one might argue that there is a motivation for the NCAA to create suboptimal seeding to make the tournament more exciting, we assert that the NCAA has a stronger motivation to reduce upsets. For the NCAA, producing accurate seedings reflect reliability as an institution. Additionally, fair seedings are important to incentivize performance and effort during the regular season by rewarding teams with an easier path in the tournament. The NCAA administration is composed of representatives chosen by colleges across the U.S., whose basketball teams participate in the tournament. To guarantee themselves a fair chance in the tournament, they are incentivized to hold the NCAA accountable for creating reliable seedings. In acknowledgment of this fact, they have a committee overseeing their complex seeding procedure [6].

There are simple heuristics for approaching these problems. When making a bracket one could, for example, choose the team with higher probability to win each matchup; similarly, one could order teams based on number of favored matchups or by average win probability to create a seeding. These naive, “greedy,” methods can find reasonable brackets and seedings, but our goal is to improve on such methods. Although it is theoretically possible to check every tournament and seeding outcome, there does not exist enough computational power or time to do so. To address this problem, our stochastic methods search only specific regions of the outcome and seeding spaces, biasing their searches towards tournaments and seeds with higher probabilities. Although these methods are not guaranteed to converge at a global optimum, they often find sufficient approximations.

Our models give compelling results for both problems we address. Our optimal bracket model outperforms a deterministic, naive baseline approach of greedily choosing the favorite in each matchup to win. We test our models using the 2023 NCAA Tournament and find that the naive approach gives a bracket with a 1.73×10^{-15} chance of occurring. Our heuristic, however, finds a better alternative with a 1.03×10^{-14} chance of occurring. That is, we improve on the naive approach by more than 6 times.

Our seeding model also outperforms a deterministic naive approach of evaluating the average win probability of each team against the tournament field and then seeds teams in descending order of this signal. Our optimal seeding beats the baseline in terms of predictive power against its most likely outcome and performs just as well at predicting the actual tournament outcome. Our seeding model’s performance is even more remarkable against the NCAA’s official seedings, which it significantly outperforms in explaining both the most likely outcome and the real tournament outcome.

We find that our statistical methods improve on several benchmarks for modeling and predicting outcomes in the NCAA Tournament. Furthermore, we find room for improvement in the NCAA’s seeding method for the 2023 tournament. By fully utilizing the information contained in matchup probability models, our statistical models prove to be effective means of modeling the NCAA March Madness tournament.

II. INTRODUCTION

The NCAA Division I men’s basketball tournament, popularly known as March Madness, offers an ideal setting for the application of stochastic modeling techniques. This tournament, a cornerstone event in American sports, features a single-elimination bracket that pits 64 of the top collegiate basketball teams against each other to determine a national champion.

Many aspects of March Madness inherently involves randomness and optimization, from the unpredictability of individual games to the challenge of optimal team seeding. Such uncertainties significantly influence the outcomes and economic implications of the event, including revenue generation. Prior studies, including those by Nelson et al. [1], Sokol [2], and Trick et al. [3], have demonstrated the importance of applying rigorous statistical methods to model and optimize game outcomes, highlighting the benefits of accurately capturing the tournament’s stochastic nature.

Our work specifically addresses two central problems associated with March Madness: predicting the most likely tournament outcomes given the initial matchups, and determining the optimal initial seeding of teams. Fans traditionally predict tournament outcomes by filling out brackets, attempting to correctly guess the outcomes of 63 matchups—a task with an astronomical number of possibilities ($2^{63} \approx 10^{18}$ possible brackets). Due to the unpredictable nature of these matchups, the probability of completing a perfect bracket is tiny, and has never been achieved. To improve prediction accuracy, more sophisticated statistical models, such as Joel Sokol’s Logistic Regression Markov Chain (LRMC) model [2] have been successfully employed, illustrating the practical value of statistical modeling by correctly predicting three of the Final Four teams in the 2025 tournament.

Similarly, the NCAA faces the complex challenge of seeding teams, ranking them from strongest to weakest to minimize “upsets”, where lower-ranked teams defeat higher-ranked opponents. The sheer complexity of this task, with $64!$ (approximately 10^{89}) possible seedings, makes finding an optimal seeding an even greater computational challenge than predicting tournament outcomes.

Our research proposes the application of advanced stochastic methods to these problems. We utilize Markov chain Monte Carlo (MCMC) techniques—particularly simulated annealing, basin hopping, and the Metropolis-Hastings algorithm—to efficiently navigate these immense solution spaces. We emphasize the limitations of simple heuristic approaches, such as greedy algorithms that always select the team with the higher probability of winning each individual matchup. Such naive approaches implicitly assume a transitive team matchup structure, where if team A is likely to beat team B, and team B is likely to beat team C, then team A is also likely to beat team C. This transitive assumption rarely holds true in the environment of sports tournaments. Consequently, naive methods often yield suboptimal predictions and seedings.

In our approach, we define specific metrics to measure the

quality of predicted outcomes and seedings. For bracket prediction, our optimal bracket model employs simulated annealing to explore likely tournament outcomes, navigating non-transitive scenarios by biasing searches towards brackets with higher probabilities. For the seeding problem, we introduce a metric based on the expected number of games that teams should win given their seeds—termed intended depth—and optimize seedings through a modified basin hopping algorithm that balances minimizing depth error and variance.

Our stochastic methods significantly outperform baseline heuristic approaches. In experiments using the 2023 NCAA tournament data, our optimal bracket model identifies an outcome approximately six times more likely than the naive, greedy approach. Similarly, our seeding optimization method substantially improves upon both naive seedings and the NCAA’s official 2023 seedings, achieving lower depth errors and providing more robust predictions of actual tournament results.

Through our work, we demonstrate the practical effectiveness of MCMC methods in addressing the inherent complexity and non-transitive nature of March Madness, providing insights for sports analysts, statisticians, and tournament organizers alike.

III. LITERATURE REVIEW

We derive our stochastic models from well-studied methodologies. Our optimization models employ Markov chain Monte Carlo methods, including simulated annealing. Simulated annealing, proposed in [7], forms the basis of our tournament outcome model. This heuristic searches for an approximate maximum value of an objective $s(x)$ by moving through the solution space according to an irreducible Markov chain. Starting in some state i , this chain proposes a state j to which the process transitions with probability $e^{-\frac{\delta}{T}}$ or probability 1 if $s(j) > s(i)$, where $\delta = s(j) - s(i)$ and T is a temperature parameter. During its run-time, simulated annealing increasingly rejects transitions towards worse states by decreasing T exponentially by a factor α .

We also employ basin hopping, an alternative Markov chain Monte Carlo Method. Basin hopping, discussed in [8], acts similarly to simulated annealing except that it uses a constant but very low temperature, T . This causes the heuristic to consistently and strongly disincentivize transitions to worse states. Augmenting its search, basin hopping introduces the possibility of random perturbations, transitions over the proposal chain that ignore the difference in objective, occurring with probability L . This perturbation allows the algorithm to escape local optima once the current optimum has been thoroughly explored. This method works better than simulated annealing for optimizing objectives with especially large numbers of deep local optima.

We also require Markov chain Monte Carlo methods that sample from the space of tournament outcomes. Because this space is so large, it is very difficult to build a representative sample. The Metropolis-Hastings algorithm allows us to do

so. Introduced in [9] and generalized in [10], the Metropolis-Hastings algorithm allows one to sample from complex distributions, known as target distributions, by enforcing a desired stationary distribution onto a Markov chain. It does so by proposing states according to a proposal chain (like the previous two heuristics). The algorithm transitions from the current state i to the proposed state j with probability

$$a(i, j) = \min(1, \frac{\pi_j T_{ji}}{\pi_i T_{ij}}),$$

where π_i is the probability of state i in the goal distribution and T_{ij} is the transition probability from state i to state j in the proposal chain. Once the chain created by the algorithm's output reaches stationarity, the time spent in each state gives a sample distribution from the goal distribution.

Markov chain Monte Carlo methods have been previously used in modeling the NCAA Tournament and finding the most likely bracket. Drawing on regression models to predict game outcomes, [1] implements a Metropolis-Hastings algorithm to choose the optimal model. Also using statistical methods, [2] predicts models the tournament outcome, successfully predicting three of the final four remaining teams for the 2025 March Madness tournament.

IV. MODEL FORMULATION & IMPLEMENTATION

Using these previously established methods and extending existing work studying March Madness, we define analytical and computational frameworks for modeling the tournament. We use these frameworks to predict tournament outcomes and optimize tournament seedings.

A. Analytical Framework

1) *Tournament Outcome Model*: The problem of finding the most likely tournament outcome given a particular set of initial matchups is equivalent to finding the mode of the distribution of all possible brackets given a set of initial matchups. Brackets are a discrete structure that can be represented as a full binary tree $B = (\mathcal{G}, \mathcal{E}, w(\mathcal{G}))$, with $\mathcal{G}$, the games played, being the nodes; $\mathcal{E}$, the possible outcomes of games, being the edges, and $w(\mathcal{G})$, the winner of each game, which is binary and a property of the node. The set of teams in the tournament we denote as $\mathcal{T}$, and we define the number of teams $n = |\mathcal{T}|$ ($n = 64$ in the case of March Madness). In this representation, each node is represented by a tuple $(i, j) \in \mathcal{T}^2$, which indicates that team i is facing team j . The leaf nodes are the initial matchups, and the parent of any two nodes is the game played between the winner of its two child node games. The root of the tree is the championship matchup. Figure 2 shows one realization of a four-team tournament with $\mathcal{T} = \{a, b, c, d\}$.

For any node in the set of tournament games $g \in \mathcal{G}$, we define a winner function $w(g)$, $g \in \mathcal{G}$, which outputs the winner of any game in the node set. We also define the function $p(i, j)$ to be the probability of team i beating team j .² Based

²In this section of the report, we assume this function to be given; in the subsequent computational section, we introduce a random forest model to estimate these probabilities using team statistics from past tournaments.

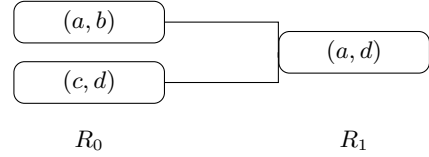


Figure 2: Example graph representation for a tournament where $n = 4$.

on these definitions, we can say that $P(w(g) = i) = p(i, j)$. Additionally, since ties are always broken in this tournament, $p(i, j) + p(j, i) = 1$, so $P(w(g) = j) = 1 - p(i, j)$.

In this representation, each layer of the tree represents a round of the tournament. We denote the set of initial matchups as $R_0 = \mathcal{L}(B)$, the leaf nodes of the binary tree.³ Subsequent rounds are determined by the winners of the previous rounds, which are denoted as $R_1, R_2, \dots, R_{\log_2(n)}$. This final round has only one element, the championship matchup. Its image under w is the set containing the tournament champion.

Based on these definitions, our goal with our tournament outcome model is to find the bracket that occurs with highest probability, given the initial set of matchups:

$$B^* = \arg \max_B P(B|R_0).$$

The set of possible brackets given an initial seeding, S , $\mathcal{B}_S$, is massive due in a 64 team tournament because each game has a separate binary outcome. In total, the tournament consists of 63 games, so $|\mathcal{B}_S| = 2^{63}$. Because $|\mathcal{B}_S|$ is so large, predicting the tournament creates an interesting mathematical challenge. Brute force search over $\mathcal{B}_S$ to find the B^* is computationally infeasible. Yet, if one wants to find the optimal solution, it is difficult to avoid. It is possible to construct adversarial examples for which a team is favored to win against every other team in the tournament individually except the team it faces first. In this example and others like it, the most likely outcome might include unlikely matchup results that are difficult to systematically identify. As a result, we introduce stochastic techniques derived from methods discussed above that account for this effect to find an approximate solution efficiently.

2) *Seeding Model*: Within our analysis of the tournament outcome problem, we find another problem we can address with stochastic methods: building the optimal tournament seeding. The outcome of a tournament is dependent on the initial matchups, which are determined by seeding. For a 64-team tournament, we denote a seeding as a tuple S with length 64, with entry i being a unique integer from 1-64 representing the seed of team i . We denote the set of all possible seedings as $\mathcal{S}$.

In a 64-team tournament the number of possible seedings for the tournament is $|\mathcal{S}| = 64! \approx 1.3 \times 10^{89}$.⁴ Furthermore,

³In this section, we assume that R_0 , which is determined by the tournament seeding, is given.

⁴We assume unique seedings and do not consider other restrictions such as region. Our methods can easily be extended to create a seeding with four of each seed from 1-16.

there are $\frac{64!}{2^{32}} \approx 3.0 \times 10^{79}$ possible arrangements of initial matchups. These astronomically large numbers once again demand specialized methods to find good seedings efficiently.

In general, a good seeding is one in which as few upsets occur as possible in expectation. However, this metric is difficult to use in optimization because it weights upsets equally regardless of their magnitude (that is, a 16-seed beating a 1-seed is weighted the same as a 9-seed beating an 8-seed). This introduces difficulty when optimizing using Markov chain Monte Carlo methods that rely on steady local descent in the objective for efficiency. To address this difficulty, we focus on the number of games a team would be expected to win based on their seed $1 \leq s \in \mathbb{Z} \leq 64$. We develop a metric we call intended depth, which we define as

$$\text{Intended Depth}(s) = id(s) = \lfloor \log_2(64) - \log_2(s) \rfloor.$$

The intended depth for seed s is the number of games that the ranking s implies that a team should win. For example, a 64-seed is expected to be eliminated in the first round and has intended depth 0, and a 1-seed is intended to win all of their games. In addition, we define a counterpart, realized depth, the number of games a team $i \in \mathcal{T}$ *actually* wins in bracket $B \in \mathcal{B}$, calculated as

$$\text{Realized Depth}_B(i) = rd_B(i) = |\{g \in \mathcal{G} : w(v) = i\}|.$$

We build an objective function for seeds based on the squared error of a team's intended depth and their expected realized depth over brackets that use this seeding (we call this conditional set of brackets B_S). An optimal seeding will minimize this squared error, which we call the depth error, defined as

$$\text{Depth Error}_S = E[\sum_{i \in \mathcal{T}} (id(S_i) - rd_{B_S}(i))^2 | S].$$

Many seedings, however, can have very similar average depth errors. Within this set of seedings with minimal squared depth error, we want the bracket that minimizes the standard deviation of the depth error. We therefore define our seeding problem as

$$\max_{S \in \mathcal{S}} -E[\text{Depth Error}_S | S] - \gamma SD[\text{Depth Error}_S | S],$$

where γ is a variance-aversion coefficient. These sample statistics can be approximated using MCMC sampling methods.

B. Computational Framework

We split our model implementation into three segments: a matchup probability framework, a bracket analysis framework, and a seeding analysis framework. Our matchup probability framework consists of our matchup probability model and a utility wrapper for this model. Our bracket analysis framework consists of a bracket object, a Metropolis-Hastings algorithm to sample bracket objects, and a simulated annealing procedure to find the most likely tournament outcome. Our seeding analysis framework consists of a seeding object and a modified simulated annealing procedure to find the optimal seeding.

1) *Probability*: We use two matchup probability functions in our experiments. The first, which we use to validate our models, is a seed-based probability model, defined by

$$P(w(i, j) = i) = \frac{S_j}{S_i + S_j}.$$

This model outputs very intuitive probabilities, making it useful when checking whether our model outputs reasonable results. For example, $P(w(i, j) = i)$ always increases as the seed of team i increases and always decreases as the seed of team j increases. As a result, we can validate our bracket optimization framework by checking that it converges to a bracket perfectly predicted by our intended depth metric. Indeed, we find that this is the case. We can similarly validate our seeding framework with this probability model by checking that seedings occur in sequential order. We find that this is the case, as well.

We define a second probability model, which we use for all other experiments. We fit a random forest classifier that predicts the winners of tournament games on ten years of historical NCAA team statistics and tournament games using the Python package scikit-learn.⁵ The outputs of our random forest model are probabilities of a given team winning a matchup against another given team.

To validate this model, we fit it on a training set and test it on an out-of-sample validation set. Our model achieves a mean squared error of 0.165 on binary labels from the validation set. Its predictions are also minimally biased as shown in Figure 3. Furthermore, we analyze the accuracy of the probabilities that our model outputs. Specifically, we want to see that, of the teams that our model gives probability x of winning, approximately x proportion of those teams win. Figure 4 shows that our model's probability assessments are roughly accurate.

We implement a wrapper for arbitrary probability models that allows us to seamlessly switch between various models in experiments. Additionally, we implement probability caching to accelerate model execution.

2) *Bracket*: Our bracket analysis framework implements utilities to apply probability models to brackets. First, we implement a bracket object that stores information about a bracket, mirroring the structure described in our analytical model formulation. This object also implements methods for the manipulation of these brackets, such as likelihood calculation, depth error calculation, and random outcome transpositions.

These utilities allow us to build a Metropolis-Hastings algorithm on the space of bracket objects (for details on the Metropolis-Hastings algorithm, see our discussion above). We implement an acceptance function that imposes the estimated likelihood of each outcome onto the stationary distribution (this function is described in more detail in Algorithm 1). We use this function to accept proposals from a Markov chain made by applying random transpositions to the current

⁵See Table I for variables and descriptive statistics of the data.

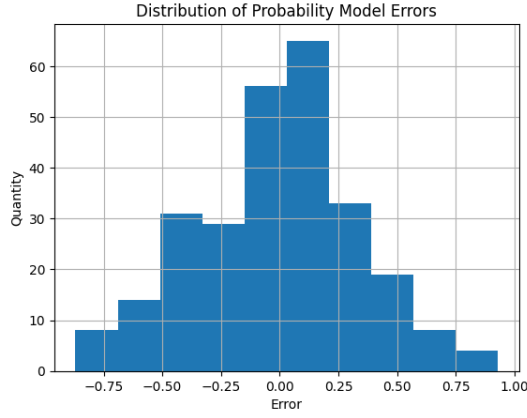


Figure 3: Our random forest classifier probability model achieves roughly unbiased probability estimates and is strongly clustered at a mean error of 0 on the validation set. Error values are calculated as the true label of the observation (0 or 1) minus the predicted probability.

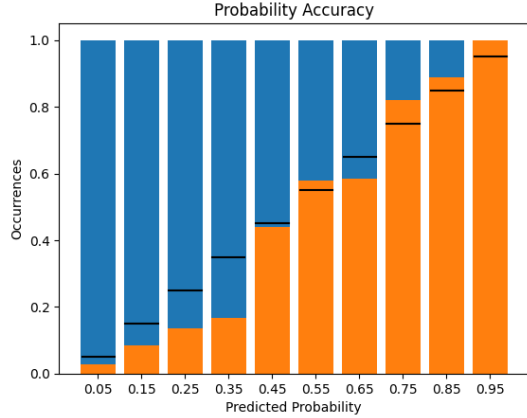


Figure 4: Our random forest classifier probability model achieves accurate probability estimates, with outcomes occurring with roughly the same prevalence as predicted. Orange bars indicate the proportion of games within each probability bin that match the label our model assigned with the binned probability value. Black lines show the theoretically expected proportion for each bin.

bracket. These random transpositions involve choosing a game at random, flipping its outcome, and propagating this new winner along the path taken by the old winner. This Markov chain is irreducible in the state space of the tournament outcomes.

Because simulated annealing resembles the Metropolis-Hastings algorithm, we reuse this framework, adding parameters T and α , and replacing the acceptance function with the simulated annealing acceptance function discussed above and detailed in Algorithm 2. To verify the consistency of our bracket optimization model, we ran several experiments on the 2023 NCAA Tournament, shown in Figure 5. From Figure 5, we can see that our simulated annealing model converges to

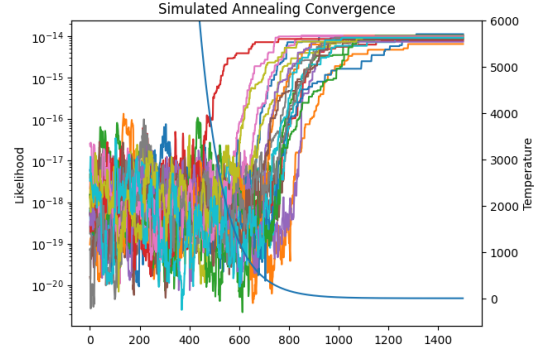


Figure 5: Our simulated annealing implementation converges to a consistent minimum. This figure shows the score of the current state in 20 tests on the 2023 NCAA Tournament, with the temperature plotted in the background. This temperature plot gives context to the annealing evident past step 400.

a consistent minimum.

As a baseline against which we can analyze our results, we define a naive approach to finding the most likely tournament outcome. In this naive approach, we proceed through the bracket, matchup-by-matchup, and choose the favorite (the team with the greatest win probability) to win each game. We continue this process through the bracket until only one team is left. We can compare our model's bracket with this bracket based on the likelihood of each to occur.

3) *Seed*: Our seeding analysis framework implements utilities to apply our bracket models to seeding optimization. First, we implement a seeding object that stores information about a seeding in the structure described in the discussion of our analytical model formulation. This object also implements methods for manipulating and evaluating these seedings, such as simulation of the implied bracket and sampling from the implied outcome distribution.

In our experiments, simulated annealing struggled to optimize our seeding objective. Hypothesizing that this failure to optimize was a result of a distribution of scores remarkably dense in local optima with no general convexity, we turned to basin hopping (detailed above). Under this approach, we propose transitions according to an irreducible Markov chain over the seed space, only accepting proposals to better states. This allows the optimizer to fully explore the local optimum it is in. In order to allow it to explore other optima, we perturb the state at random intervals. These perturbations involve randomly shuffling the seeds of between 2 and 10 teams separated from each other by no more than 5 overall spots. We find that this approach gives significant improvements on simulated annealing. We discuss this finding more in the Computational Findings section.

In order to estimate the objective function for a seeding, we use our Metropolis-Hastings algorithm on the bracket space to sample outcomes according to the seeding. We find that this method gives consistent estimates of the objective for a given seeding, as Figure 6 shows.

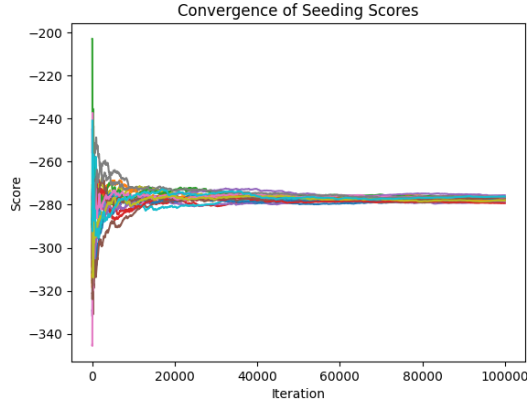


Figure 6: Using the NCAA’s official seedings for the 2023 NCAA Tournament, we find that our model converges to a reliable estimate of the seeding objective function.

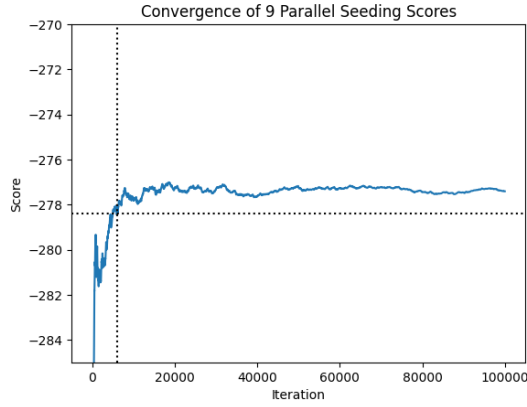


Figure 7: The intensity of the outcome sampling task requires computational optimizations. As a result, we parallelize 9 outcome chains and average their estimates of the objective. The vertical dotted line indicates the iteration at which we terminate sampling, and the horizontal dotted line indicates the objective estimate at this iteration. Although not finding the exact converged value, this method approximates it sufficiently well, given our computational limitations.

This process is computationally intensive, requiring significant optimizations in order to terminate in a reasonable amount of time. As a result, instead of sampling from one chain, as shown in Figure 6, we sample from 9 chains in parallel and average their results. Although we get fewer samples in stationarity, we still find that this method gives us a reliable estimate of the objective, as shown in Figure 7.

Under our seeding procedure, we run our seeding model for 40,000 iterations and save all states visited. We then take the top 20 unique states found during this execution and calculate a more reliable estimate of the objective using a larger sample, with more items in stationarity. The state with the maximum objective by this estimate is given as the output as our seeding model.

As a baseline against which we can compare our seeding

model, we define a naive approach. In this naive approach, we consider every possible matchup that can occur (there are $\frac{63 \cdot 62}{2}$ possibilities). We calculate the probabilities of each matchup outcome, then find the average chance that each team beats each other team. We then sort the teams by this score and assign seedings, with teams with a higher average win chance getting a better seed. We also use the NCAA’s official seeding as a benchmark.

We can compare our seedings to these benchmarks using the likelihood of the most likely bracket found under the seedings as a metric for the seedings’ stability. We can also compare on the basis of the depth error metric that we defined above. We can also use a similar metric called outcome error to measure the seeding’s predictive power against the true tournament outcome. We define outcome error as

$$\text{Outcome Error}_S = E\left[\sum_{i \in \mathcal{T}} (td(S_i) - rd_{B_S}(i))^2 | S\right],$$

where $td(i)$ is the true number of wins that the team actually achieved in the tournament.

V. RESULTS

Our stochastic models substantially improve on our benchmarks. For the optimal bracket problem, our model improves on the naive approach by more than six times. For the optimal seeding problem, our model improves on the naive approach by 30 units of depth error. Furthermore, our model improves on the NCAA’s official seedings by 28 units of depth error and by 10 units of outcome error.

A. Analytical Findings

1) *Optimal Brackets:* To simplify our analytical approach, we make the reasonable assumption that games within each round are independent. We can justify this assumption, because teams only play once per round, and games in the same round often occur at the same time. Thus, the probability that any given round will occur depends only on the probabilities of the individual matchups within that round. We can also assume without loss of generality that, for a game (i, j) , team i is the winner under the bracket since ordering is irrelevant. Based on these assumptions, the overall probability that a given bracket $B = (\mathcal{G}, \mathcal{E}, w(\mathcal{G}))$ will occur is given by the product of the probabilities of each outcome in the bracket:

$$P(B) = \prod_{(i,j) \in \mathcal{G}} p(i, j).$$

One might expect that the bracket with the highest probability of occurring is the one which always chooses the team with the higher probability of winning. Consider the following proposition:

Proposition 1. *Suppose $B^* = (\mathcal{G}, \mathcal{E}, w(\mathcal{G}))$ is an optimal bracket for a tournament where all matchups are transitive. That is, $P(B^*) \geq P(B)$ for all $B \in \mathcal{B}$ and for any three teams x, y , and z , if $p(x, y) > 0.5$ and $p(y, z) > 0.5$ implies that $p(x, z) > 0.5$. Then for all games $(i, j) \in \mathcal{G}$, $w(i, j) = i$ only if $p(i, j) > 0.5$.*

This bracket is the product of the naive approach to creating a bracket. If team i is the favorite to win against team j , ($p(i, j) > 0.5$), the bracket always chooses that team. We will prove this proposition under the assumption that team strengths are transitive.

Proof. Assume that in an optimal bracket B^* there exists a game $g = (i, j)$ where the bracket predicts $w(i, j) = i$ but $p(i, j) < 0.5$. Then by definition of our win probability, $p(j, i) = 1 - p(i, j) > 0.5$.

Now, suppose we construct an alternative bracket B' that is identical to B^* except that in game g we swap the outcome by setting $w'(i, j) = j$. Since the tournament is structured so that outcomes propagate downstream, we update all subsequent games in the branch affected by g in a consistent manner. Under our transitivity assumption, team j 's chances of winning in the subsequent game are at least as good as those of team i , so the probability of the branch following game g does not decrease when switching from team i to team j .

Let $P(B'_g)$ denote the contribution to the overall bracket probability from the subtree originating at game g in B' and let $P(B_g^*)$ be the corresponding branch probability in B^* . Then, by transitivity, we have $P(B'_g) \geq P(B_g^*)$ since all downstream games only have the potential to increase in probability by picking team j where i once was. Moreover, because the immediate probability in game g has improved upon switching (since $p(j, i) > p(i, j)$), it follows that $p(j, i) \cdot P(B'_g) > p(i, j) \cdot P(B_g^*)$.

Since the overall probability of a bracket is the product over all branches, replacing the outcome in g (with its downstream effects) increases the overall probability: $P(B') > P(B^*)$, contradicting the optimality of B^* .

Thus, for any game $g = (i, j)$ in an optimal bracket B^* having $p(i, j) > 0.5$, it must be that the predicted winner of that game is $w(i, j) = i$. $\square$

This result tells us that if we are confident in assuming transitivity, we can greedily choose the team with the higher winning probability in each pairing to create an optimal bracket.

To better understand what happens without the transitivity assumption, we will consider an example tournament with four teams labeled A, B, C, D . Define the probability matrix $\mathbf{P}$ with $P_{ij} = p(i, j)$ as

$$\mathbf{P} = \begin{matrix} & \begin{matrix} A & B & C & D \end{matrix} \\ \begin{matrix} A \\ B \\ C \\ D \end{matrix} & \begin{pmatrix} - & 0.49 & 0.99 & 0.98 \\ 0.51 & - & 0.48 & 0.47 \\ 0.01 & 0.52 & - & 0.46 \\ 0.02 & 0.53 & 0.54 & - \end{pmatrix} \end{matrix}$$

and suppose that, in the first round, team A faces team B and team C faces team D (as shown in Figure 8). Note that because $p(i, j) + p(j, i) = 1$, this matrix has the property that $\mathbf{P} + \mathbf{P}^T = \mathbf{J}$ (the one's matrix). We can see that $p(C, B) = 0.52 > 0.5$ and $p(B, A) = 0.51 > 0.5$, and yet $p(A, C) = 0.99$, so this tournament exhibits non-transitivity.

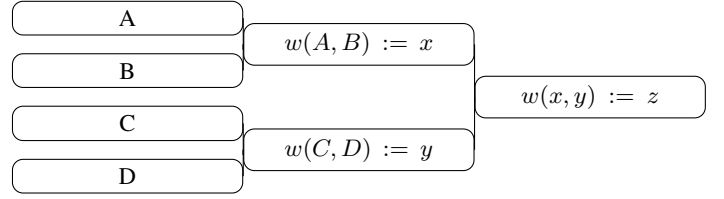


Figure 8

Under this example, there are three total games played, yielding a total of 2^3 possible brackets. A bracket in this tournament can be uniquely denoted as $(w(A, B) = x, w(C, D) = y, w(x, y) = z)$. We can enumerate the probabilities of each possible bracket:

$$\begin{aligned} P(w(A, B) = A, w(C, D) = D, w(A, D) = A) &= 0.259308 \\ P(w(A, B) = A, w(C, D) = C, w(A, C) = A) &= 0.223146 \\ P(w(A, B) = B, w(C, D) = D, w(B, D) = D) &= 0.145962 \\ P(w(A, B) = B, w(C, D) = D, w(B, D) = B) &= 0.129438 \\ P(w(A, B) = B, w(C, D) = C, w(B, C) = C) &= 0.121992 \\ P(w(A, B) = B, w(C, D) = C, w(B, C) = B) &= 0.112608 \\ P(w(A, B) = A, w(C, D) = D, w(A, D) = D) &= 0.005292 \\ P(w(A, B) = A, w(C, D) = C, w(A, C) = C) &= 0.002254. \end{aligned}$$

We observe that the highest probability bracket where $w(A, B) = A, w(C, D) = D, w(A, D) = A$ has team A as the winner of its first match, despite the fact that team B is favored to win, as $p(A, B) = 0.49 < 0.5$. This shows that, without the transitivity assumption, we cannot use a greedy approach.

In general, transitivity in the context of sports modeling is a poor assumption. A team may possess certain characteristics that make them particularly effective against a generally strong team while also struggling against another generally weaker team. For example [11] finds empirical evidence of non-transitivity in English Premier League soccer games. Although soccer is not basketball, the dynamics of a team sport discussed are, nevertheless, applicable. This result suggests that non-transitivity may also be present in March Madness.

We can clearly see within the 2023 March Madness bracket data that our random forest model exhibits non-transitivity. The following matrix defines the probability function $p(i, j) = P_{ij}$ for Alabama, Kansas, and San Diego State:

$$\mathbf{P} = \begin{matrix} & \begin{matrix} A & K & S \end{matrix} \\ \begin{matrix} A \\ K \\ S \end{matrix} & \begin{pmatrix} - & 0.495 & 0.51 \\ 0.505 & - & 0.485 \\ 0.49 & 0.515 & - \end{pmatrix} \end{matrix}$$

We can see that this probability function violates the definition of transitivity in this case as $P(K = w(K, A)) > 0.5$ and $P(A = w(A, S)) > 0.5$, yet $P(K = w(K, S)) = 0.485 < 0.5$. Thus, we have a clear example of random forest exhibiting non-transitivity.

Without the result of Proposition 1, there is no obvious way to address non-transitivity short of using the brute force method. In a 64-team tournament, the 2^{63} possible brackets make this approach computationally infeasible. We turn to stochastic models to implement a more efficient search. To test this methodology, we apply our model to this non-transitive example. We find that our model reliably converges to the optimum.

2) *Optimal Seeding*: Similar to its effect on finding the most likely tournament outcome, non-transitivity introduces challenges into seeding the tournament as well. A tournament satisfying transitivity is trivial to seed once you have a matchup probability model, because the strongest team is guaranteed to be stronger than *all* the other teams, yielding a clear one-seed. This logic proceeds for each other seed.

For a non-transitive set of teams, seeding becomes more complicated. If, among the top three teams, A , B , and C , A beats B , B beats C , and C beats A on average, it is much less clear which team ought to be the first overall seed. The naive approach might be to pick the team that is favored to win in the greatest number of matchups among all the teams in the tournament. Another potential method would be to choose the team with the greatest average win probability across all possible matchups, which might give a more refined seeding by accounting for close matchups. For a transitive set of teams, both of these methods can be performed in quadratic time with respect to the number of teams and will successfully find the optimal seeding. Proof of this statement follows similarly from the proof for finding the most likely bracket. However, in the presence of non-transitivity, these algorithms may once again fail.

To prove this fact, we will introduce a new counterexample. Consider another tournament with teams $A - D$ with matchup probabilities given by

$$\mathbf{P} = \begin{matrix} & \begin{matrix} A & B & C & D \end{matrix} \\ \begin{matrix} A \\ B \\ C \\ D \end{matrix} & \begin{pmatrix} - & 0.75 & 0.6 & 0.9 \\ 0.25 & - & 0.65 & 0.85 \\ 0.4 & 0.35 & - & 0.2 \\ 0.1 & 0.15 & 0.8 & - \end{pmatrix} \end{matrix}.$$

Under this example, A is clearly the strongest team overall, B is second-best, and D dominates C head-to-head with win probability of 0.80 even though D is weak versus A and B . This makes the relative order of C and D the key seeding decision.

Now, instead of assuming the initial matchups are fixed as we do in our tournament outcome model, we allow seedings to vary. Here we will denote a seeding uniquely as an ordering of the teams, e.g. $BCDA$ means that team B is the top seed and A is the bottom seed. Under the first naive method of seeding the team with the largest number of favorable matchups, we see that A is favored to win all matchups, B is favored to win two of three matchups, C is favored to win no matchups, and D is favored to win one of three matchups. Thus, this approach produces seeding $ABDC$. If we instead go by average win

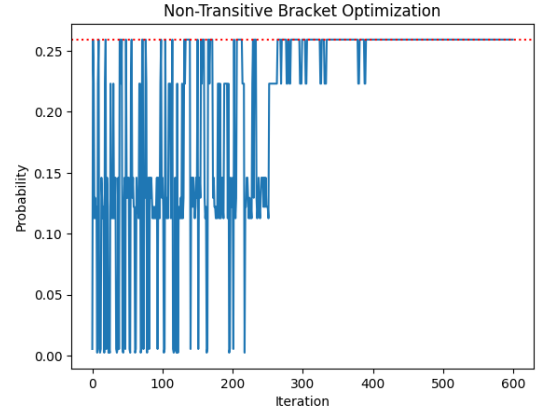


Figure 9: Our simulated annealing model successfully finds the optimal solution to our analytical example. The blue line shows the state of the simulated annealing algorithm at each iteration, and the red dotted line is the objective value of the optimal solution. For this example, we use $\alpha = .99$ and $T = 10^{29}$. We show all steps to emphasize that the model converges to the optimal solution, beyond simply discovering it because discovery is nearly guaranteed in this state space of eight states.

probability, we get that A has average win probability 0.75, B has average probability 0.583, C has average probability 0.317, and D has average probability 0.35, once again yielding seeding $ABDC$.

Under this seeding, the round one matchups are team A vs. C and B vs. D , and the winners face off for the championship. We can compute the expected number of upsets for this seeding by enumerating the number of upsets for each possible outcome, multiplying by the outcome probability, and summing. This gives us that the expected number of upsets under this seeding is 0.8175. However, if we perform the same calculation for seeding $ABCD$ (taking the new initial matchups into account), we get a value of 0.739 expected upsets, demonstrating that the naive approaches were suboptimal. This provides motivation for introducing stochastic methods for finding optimal seedings, which can yield improvements over the naive approach while still beating the brute force approach in runtime.

B. Computational Findings

As a first example of our computational results, we will optimize the non-transitive example from our analytical discussion. Our model successfully finds the optimal solution to this example, as shown in Figure 9. It first finds this example early in its exploration then converges to it by iteration 450.

For the problem of finding an optimal bracket for a larger set of games, we compute a naive result and our model result. Our model uses the simulated annealing parameters $T_0 = 500000$ and $\alpha = 0.99$. In order to accelerate our model, we cache our probability model's outputs. We are able to run this model on an M2 MacBook Pro. When the cache is empty, optimization

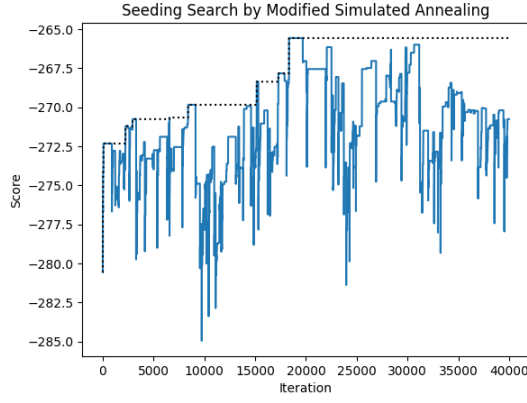


Figure 10: Our basin hopping model searches for the optimal seeding with a hybrid basin-hopping, simulated annealing approach. The blue line shows the score of the current state, while the black dotted line shows the best achieved score.

takes approximately 167 seconds. After populating the cache, optimization takes approximately 1 second.

Using our random forest probability model, our naive approach finds an outcome with a 1.73×10^{-15} probability (see Figure 17 for the naive predictions). Our simulated annealing model finds an outcome with a 1.03×10^{-14} (see Figure ?? for our model’s predictions), improving on the naive results by more than six times.

Addressing the optimal seeding problem, we compute a naive result and our model result. Our model uses parameters $T = .2$ and $L = \frac{1}{500}$. This causes the model to follow a time-invariant and mostly stable descent except for perturbations at one in five hundred steps. These perturbations occur as discussed in our model formulation discussion. We run our model for 40,000 steps, taking 26 hours and 48 minutes.

The loss graph for our model execution (Figure 10) illustrates the model’s exploration for the optimal seeding. The graph shows long periods of stagnation interspersed with significant disturbances from perturbations followed by resettlement.

We test alternative approaches to the seeding problem using standard simulated annealing with various choices for α and T . Figure 11 shows these experiments alongside our approach. Our approach spends more time in states with better objective scores and eventually reaches a better approximation of the solution. It also finds more viable candidate solutions, an important difference given that our method tests several candidates to find its proposed seeding.

In experiments with higher α values and no perturbations, the model quickly diverges by taking many backwards steps with low δ . Transpositions that cause these changes are much more common than transpositions that cause score improvement. Figure 12 shows a more detailed view of our model’s optimization, which indicates the rarity of score-improving transpositions. In the red box (Figure 11), we see that many resettlements that appear to be rapid in the original plot are actually made up of several smaller discrete improvements.

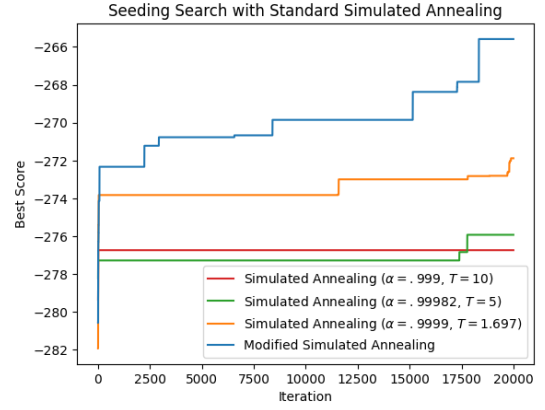


Figure 11: Our method spends more time in states with higher objectives and is able to probe deeper into areas of higher objectives than simulated annealing. All other simulated annealing approaches plateau far behind our model’s solution.

In the green box, we see that these improvements are spaced out significantly and are connected by significant periods of stagnation.

These stagnant periods are a result of our acceptance function rejecting transitions that cause regressions in our score function. If our acceptance function accepted even a small fraction of these regressions, as it would in unmodified simulated annealing, the model would never have the ability to converge to a reasonable solution in the number of steps used. Such a method would either regress too much even with a low temperature given the rarity of improving transitions or never have a chance to fully escape its local optimum.

Our model finds a seeding with an objective score⁶ of -270.09 and a most likely tournament outcome with depth error 99, outcome error 138, and probability 1.84×10^{-14} . Notably, this outcome is almost twice as likely as the most likely outcome we find based on the NCAA’s seedings when we address the tournament outcome problem. This suggests that our seeding arranges games such that there are fewer matchups where the probability of either team winning is close, minimizing upsets.

Using a naive approach, we find a seeding with an objective score of -281.23 and a most likely tournament with depth error 129, outcome error 138, and probability 5.67×10^{-15} . We show the differences between our model’s seedings and the naive seedings in Figure 13. Our model improves on the naive objective by more than 11 points and the naive depth error by 30 points.⁷

⁶This value does not match the value shown in the score graph because the optimization function relies on a more variable estimate of the objective. To account for this variability, we choose our optimal seeding by evaluating the top 20 candidates from our model. See Model Formulation above for more discussion of this technique.

⁷This depth error change, considering the definition of depth error above, is the equivalent of predicting about five more individual matchups correctly ($5^2 = 25$) or two more four-round runs correctly ($2 \times 4^2 = 32$). The objective is more difficult to interpret, as it combines error and variability in a way that cannot be disentangled.

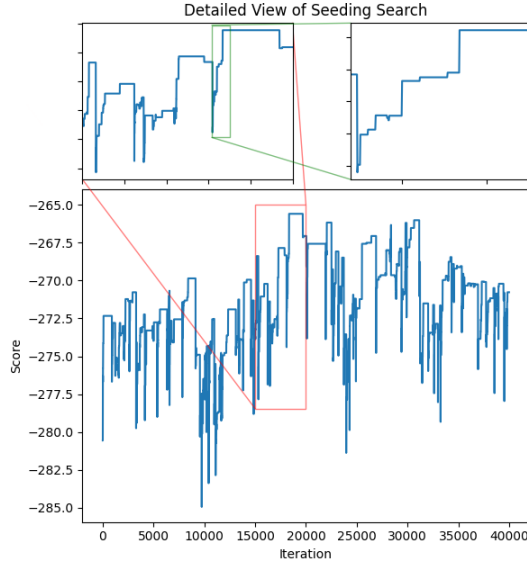


Figure 12: We highlight a few more detailed views of our seeding optimization. In the upper left, we show a subset of the optimization to give more context to the alternation of stagnations, perturbations, and resettlements. In the upper right, we show that a resettlement is actually a sequence of small, rare improvements, interspersed with significant stagnations.

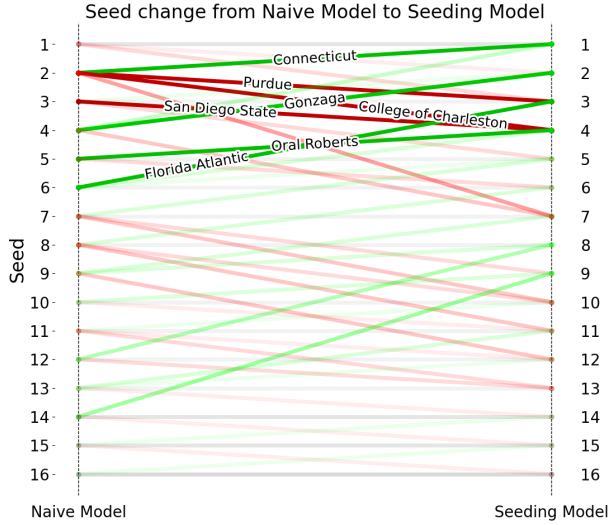


Figure 13: Our model makes several changes to the naive model, including increasing eventual champion Connecticut's ranking and decreasing 1-seed Purdue's ranking, which became the first 1-seed to lose in the first round. This figure shows those changes with the naive results plotted on the left and our model's results on the right. Lines show the seed change for each team with color indicating the direction of the change and intensity indicating the magnitude of the change.

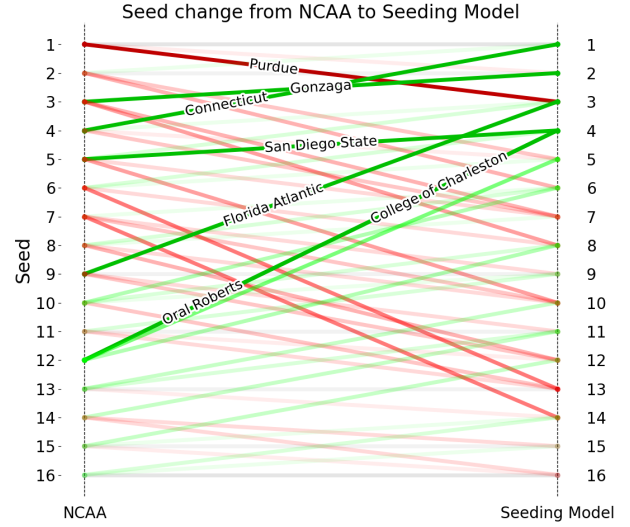


Figure 14: Our model makes many significant changes to the NCAA's official seedings. Eventual champion Connecticut improves to a 1-seed and underdog Final Four competitor Florida Atlantic improving to a 3-seed. This figure shows those changes with the NCAA's official seedings plotted on the left and our model's results on the right. Lines show the seed change for each team with color indicating the direction of the change and intensity indicating the magnitude of the change.

Using the NCAA's seedings, we find an objective score of -279.15 and most likely tournament outcome with depth error 127 and outcome error 148.⁸ We show the differences between our model's seedings and the NCAA's seedings in Figure 14. Our model improves on the NCAA's seedings by all measures. We improve the objective by more than 9 points, the depth error by 28 points, and the outcome error by 10 points.⁹

We can also analyze the differences between seedings qualitatively to illustrate the improvements and regressions of our model's optimal seed.

One of the most obvious observations is that the NCAA ranks the tournament's actual winner, University of Connecticut, as a 4-seed. Our naive approach, on the other hand, places the champion at a 2-seed, and our model places them even higher as a 1-seed. We also look at one of the most notable underdogs from the tournament, Florida Atlantic University. The NCAA seeded the team, which made it to the Final Four at a 9-seed (implying that they should have lost in the first round). Our naive approach recognizes some potential in the team, placing them at a 6-seed (expected to win 1 game), while

⁸Finding the probability of this outcome is the same as addressing the tournament outcome problem above, so we do not discuss it again here.

⁹The interpretations of these improvements are the same as for the naive approach. The change in outcome error, for example, is equivalent to predicting three more individual matchups correctly or one more three-round run correctly.

our model is even more confident, placing them at a 4-seed (expected to win 3 games). In another notable outcome from this tournament, Purdue became the first ever 1-seed to lose to a 16-seed. The naive approach recognized Purdue as slightly overrated, placing them at a 2-seed, while our model identified them as more overrated, placing them at the last 3-seed.

At times, our model reversed the naive approach’s adjustments to NCAA rankings. The NCAA ranked San Diego State as a 5-seed, but the team survived until the championship game. Our naive approach recognized the San Diego State might be underrated, moving them up to a 3-seed. Our model disagreed, however, and moved them back down to the last 4-seed. Our model also disagreed with the naive approach’s assessment of Gonzaga. The NCAA seeded Gonzaga as a 3-seed, but our naive approach seeded them as a 4-seed. Our model moved them up to a 2-seed. In the tournament, Gonzaga performed exactly as a 2-seed should, making it to the Elite Eight by beating its region’s 2-seed, UCLA, before losing to the tournament’s eventual champion.

However, our model overestimated some underdogs. The NCAA seeded Oral Roberts and College of Charleston as 12-seeds. Our naive approach seeded them at a 5-seed and a 2-seed, respectively, and our model seeded both as 4-seeds. Both teams lost in the first round, as the NCAA’s seed would imply they should.

We understand that these anecdotal comparisons and narratives obscure the probabilistic aspect of our stochastic models. Still, we believe that this analysis is useful as it checks the plausibility of our results and lends extra tangibility to the theoretical measures and methods that we have used until this point.

VI. CONCLUSIONS AND RECOMMENDATIONS

Ultimately, our optimal tournament outcome model outperforms the naive approach, finding an outcome that is 6 times more likely. Our optimal seeding model outperforms both the naive seeding approach and the official seeds of the 2023 NCAA March Madness tournament, evidenced both with metrics we develop and qualitative analysis.

We address the problem of finding the most likely outcome of a given tournament using simulated annealing and the problem of finding the optimal seed using basin hopping. Furthermore, to facilitate our second analysis, we use the Metropolis-Hastings algorithm to sample from the set of outcomes of a given tournament. This allows us to calculate descriptive statistics for various metrics of outcomes.

Through these analyses, we show that Markov chain Monte Carlo methods are effective means of using a non-transitive probability model for matchup outcomes. On top of accounting for non-transitivity and improving on existing approaches, our methods make computational optimizations that allow them to execute with the limited computational power available to an average consumer. These lightweight and effective stochastic solutions offer significant utility to our target audiences.

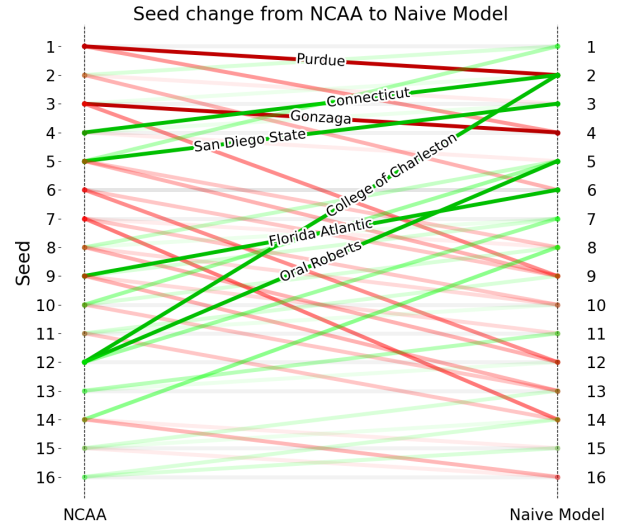


Figure 15: The naive model differs from the NCAA seeding, improving eventual champion Connecticut’s ranking by two seeds. Our subsequently raises Connecticut’s ranking another seed. This figure shows those changes with the NCAA’s official seedings plotted on the left and the naive model’s results on the right. Lines show the seed change for each team with color indicating the direction of the change and intensity indicating the magnitude of the change.

REFERENCES

- [1] B. T. Nelson, “Modeling the ncaa tournament through bayesian logistic regression,” Master’s thesis, Duquesne University, 2012.
- [2] J. Sokol, “Profspicks: Play along with professors,” 2025. Accessed: 05.03.2025.
- [3] G. L. Nemhauser and M. A. Trick, “Scheduling a major college basketball conference,” *Operations Research*, vol. 46, no. 1, pp. 1–8, 1997.
- [4] ESPN, “Espn bet men’s college basketball futures,” 2025. Accessed: 05.08.2025.
- [5] NCAA, “2023 d1 men’s basketball championship official bracket,” 2023. Accessed: 04.09.2025.
- [6] NCAA, “Building the brackets: A deep dive on the ncaa tournament selection and seeding process,” 2025. Accessed: 04.29.2025.
- [7] S. Kirkpatrick, C. D. Gelatt Jr, and M. P. Vecchi, “Optimization by simulated annealing,” *science*, vol. 220, no. 4598, pp. 671–680, 1983.
- [8] Y. O. Masao Iwamatsu, “Basin hopping with occasional jumping,” *Chemical Physics Letters*, vol. 399, pp. 396–400, 2004.
- [9] N. Metropolis, A. W. Rosenbluth, M. N. Rosenbluth, A. H. Teller, and E. Teller, “Equation of state calculations by fast computing machines,” *The Journal of Chemical Physics*, vol. 21, no. 6, pp. 1087–1092, 1953.
- [10] W. K. Hastings, “Monte carlo sampling methods using markov chains and their applications,” *Science*, 1970.
- [11] J. Van Ours, “Non-transitive patterns in sports match outcomes: A profitable anomaly,”

APPENDICES

Feature	Mean	Std	Min	Med	Max
Away Losses (Team 0)	4.8	2.4	0.0	5.0	17.0
Conference Losses (Team 0)	4.9	2.4	0.0	5.0	13.0
Conference Wins (Team 0)	12.6	2.4	5.0	13.0	19.0
Games Played (Team 0)	35.8	2.1	30.0	36.0	41.0
Home Wins (Team 0)	14.2	2.2	5.0	15.0	20.0
Losses (Team 0)	9.2	3.3	1.0	9.0	21.0
Pace (Team 0)	67.2	3.2	57.3	67.1	77.3
Simple Rating System (Team 0)	14.3	7.8	-13.4	15.5	28.7
Turnover Percentage (Team 0)	15.1	1.6	10.6	15.0	20.7
Wins (Team 0)	26.7	4.5	14.0	26.0	38.0
Block Percentage (Team 1)	11.0	2.9	3.8	10.7	20.4
Conference Losses (Team 1)	4.9	2.4	0.0	5.0	13.0
Games Played (Team 1)	35.8	2.1	30.0	36.0	41.0
Home Wins (Team 1)	14.2	2.2	5.0	15.0	20.0
Losses (Team 1)	9.2	3.3	1.0	9.0	21.0
Opp Offensive Rebound Percentage (Team 1)	29.9	3.5	20.8	29.6	42.8
Steal Percentage (Team 1)	9.9	1.7	5.5	9.7	17.0
Strength Of Schedule (Team 1)	5.8	5.0	-12.3	7.6	12.8
Wins (Team 1)	26.7	4.5	14.0	26.0	38.0
Winner	0.5	0.5	0.0	0.5	1.0

Table I: Probability Model Training Data Summary Statistics



Figure 16: Our model finds this approximation of the most likely tournament outcome with a 1.03×10^{-14} chance of occurring.

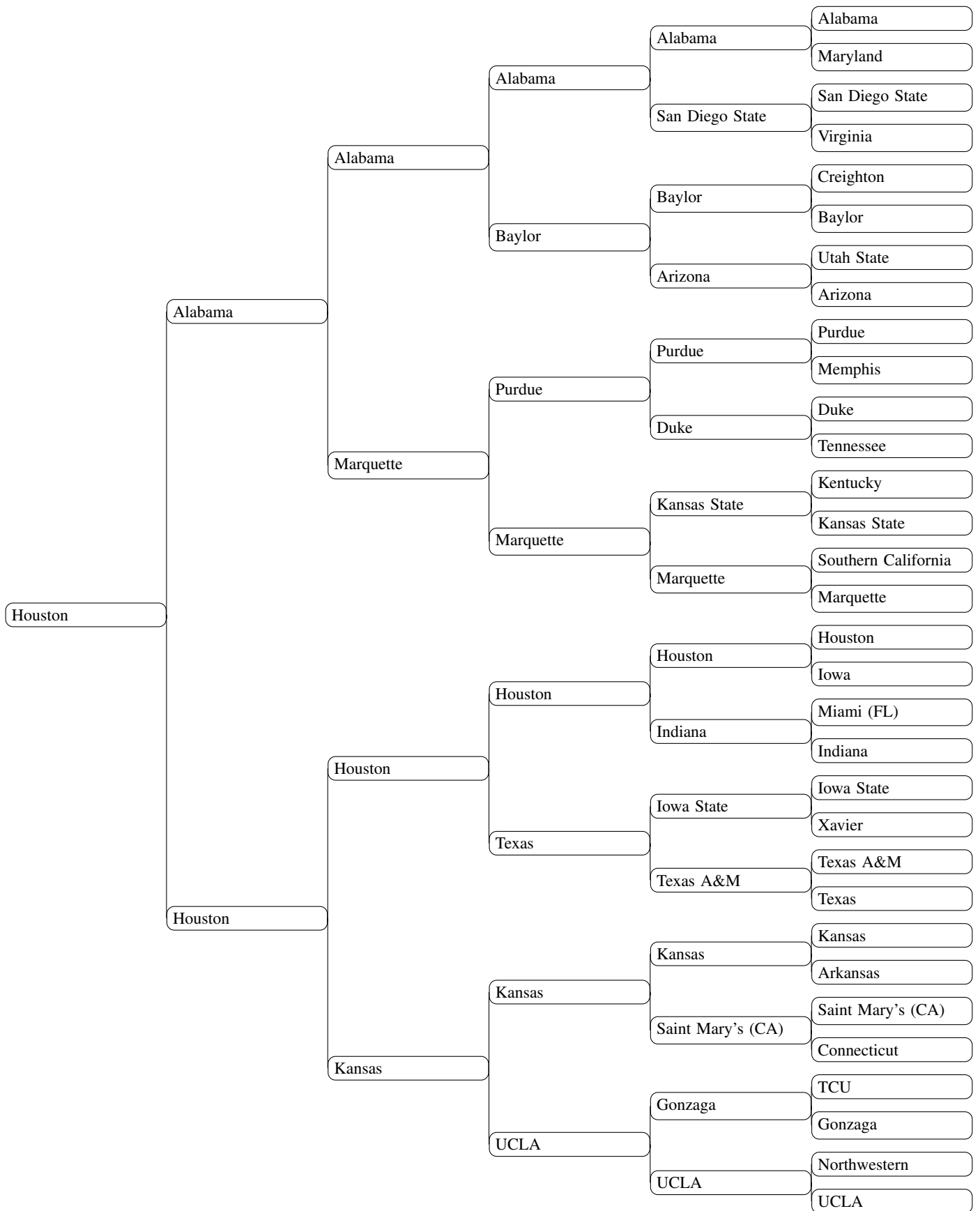


Figure 17: The naive approach finds this approximation of the most likely tournament outcome with a 1.73×10^{-15} chance of occurring.



Figure 18: The real outcome of the 2023 NCAA Division I Men's Basketball Tournament provides context for our model's results.

Algorithm 1 Metropolis Hastings Acceptance Function

```
function ACCEPT(bracket1, bracket2)
   $p \leftarrow P(\text{bracket1}) \div P(\text{bracket2})$ 
  if  $p > 1$  then
    return bracket1
  else
     $u \sim U(0, 1)$ 
    if  $u < p$  then
      return bracket1
    else
      return bracket2
```

Algorithm 2 Simulated Annealing Acceptance Function

```
function ANNEAL_ACCEPT( $\text{bracket1}, \text{bracket2}$ )
   $\delta \leftarrow P(\text{bracket1}) - P(\text{bracket2})$ 
  if  $\delta > 0$  then
    return bracket1
  else
     $u \sim U(0, 1)$ 
    if  $u < e^{\delta/T}$  then:
      return bracket1
    else
      return bracket2
```
